

Contents

Mermaid Diagram Plan	1
1. Conventions this plan assumes	1
2. Diagrams that already exist	2
3. Front matter — 00-front-matter.md	2
4. Chapter 0 — Orientation (01-orientation.md)	4
5. Chapter 1 — Meet Your Coding Agent (02-meet-your-coding-agent.md)	5
6. Chapter 2 — Writing SPEC.md (03-writing-spec.md)	6
7. Chapter 3 — Holding the Agent to a Standard (04-ruff.md)	7
8. Chapter 4 — The Domain: Fixed Mortgages (05-fixed-mortgages.md)	8
9. Chapter 5 — Tests & Test-Driven Development (06-tests-and-tdd.md)	10
10. Chapter 6 — Mathematical Core Library (07-math-core-library.md)	12
11. Chapter 7 — Validation (08-validation.md)	13
12. Chapter 8 — Command Line Interface (09-cli.md)	14
13. Chapter 9 — Calculator UI (10-calculator-ui.md)	15
14. Chapter 10 — Tool Interface (11-tool-interface.md)	16
15. Chapter 11 — LLM Interface (12-llm-interface.md)	18
16. Chapter 12 — Evaluation (13-evaluation.md)	21
17. Chapter 13 — Hardening (14-hardening.md)	22
18. Closing (15-closing.md)	23
19. Appendix (16-appendix.md)	24
20. Suggested build order	25

Mermaid Diagram Plan

A chapter-by-chapter proposal for where this book would read better with a diagram, what kind of diagram, and what it should actually show.

Scope: the manuscript as of `manuscript/00-front-matter.md` through `manuscript/16-appendix.md`. Four mermaid diagrams already exist; they are listed below as-is, and the chapters holding them are still reviewed for whether *more* would help.

1. Conventions this plan assumes

Fence and build note. Every diagram goes in a plain ````mermaid` fence, followed by the same HTML comment the existing four carry:

```
<!-- DIAGRAM BUILD NOTE: render this mermaid block to an image (e.g. via mermaid-cli) for the print/PDF
```

Both builds already handle this automatically — `tools/build-book-html.sh` turns on `--filter mermaid-filter` with `MERMAID_FILTER_FORMAT=svg` and `tools/build-book-localtoc.sh` uses `MERMAID_FILTER_FORMAT=pdf` (vector, scale 3) — but the comment stays for anyone building the Markdown by hand.

Diagram types that are safe here. `flowchart`, `sequenceDiagram`, `stateDiagram-v2`, `timeline`, and `classDiagram` are all well-supported by the mermaid-cli version this repo already renders with (`timeline` is proven — Chapter 13 uses one). Anything `-beta` (`xychart-beta`, `block-beta`, `sankey-beta`) and `mindmap` are **version-sensitive**: they are proposed below only where they clearly beat the alternatives, and each one needs a `make book` and `make book.html` check before it lands.

Label quoting. Any label containing a comma, colon, parenthesis, or % needs double quotes — `D["Context Window: how many tokens it can see"]` — the way the Chapter 1 and 13 diagrams already do it.

Width. Prefer `flowchart LR` for two-to-four-stage pipelines and `flowchart TD` for anything with branching, so nothing overflows the PDF's text block. A pipeline longer than four boxes should wrap to TD.

The bar for including one. This book's prose is dense and careful, and a diagram that only restates a paragraph makes the page longer without making it clearer. The diagrams proposed below all do one of three things the prose can't do cheaply: show a **branch** (two or more outcomes from one point), show a **loop** (something that returns to where it started), or show **convergence** (several paths meeting at one place). Where a chapter's existing ASCII art already does the job — Chapter 0.7.3's project tree, Chapter 9.4.1's window sketch, Chapter 11.9.2's `docs/ui.md` layout — this plan explicitly leaves it alone.

Priorities. **P1** = the chapter is materially clearer with it; build these first. **P2** = worthwhile, clearly earns its space. **P3** = nice to have, drop it if the chapter is running long.

2. Diagrams that already exist

Chapter	File	§	Type	What it shows
1	02-meet-your-coding-agent.md		<code>flowchart TD</code>	Language model → tokens → next-token prediction → context window → local vs. hosted
13	14-hardening.md	13.3.1	<code>flowchart LR</code>	The four seams converging on <code>MortgageInput</code> , with the model's path routed through a parse step first
13	14-hardening.md	13.5.2	<code>flowchart LR</code>	One <code>logger</code> call fanning out to <code>stderr</code> (INFO) and a rotating file (DEBUG)
13	14-hardening.md	13.6	<code>timeline</code>	SPEC.md's five revisions across Chapters 2, 4.7, 9.6.2, 11.9.2, 13.6

Note the imbalance this plan is mostly correcting: Chapter 13 has three diagrams and Chapters 4–12 — where the actual system gets built — have none.

3. Front matter — 00-front-matter.md

§	Type	Diagram	Priority
“What ‘Hybrid’ Means in This Book”	flowchart TD	The hybrid boundary. Two labelled zones. <i>Yours</i> : SPEC.md, the worked example, tests, the eval set, every review decision. <i>AI-assisted</i> : Pi drafting implementations, the intelligence-engine model answering questions. A single arrow crosses from Yours → AI-assisted labelled “what ‘correct’ means”, and no arrow crosses back. This is the book’s actual thesis and it currently exists only as prose in the front matter and again in C.4 — a picture stated once here and echoed in the Closing would make the whole book feel like one argument.	P1
“How This Book Is Structured”	flowchart LR or timeline	The braid. The two threads running side by side: the <i>tools</i> thread (terminal/git → agent → Ruff → pytest → Pydantic → argparse → PyQt5 → Ollama/OpenRouter → loguru) above, the <i>system</i> thread (empty repo → spec → core → validation → CLI → GUI → tool → LLM → eval → hardened) below, with each tool tied to the chapter that needs it. Gives a reader the whole arc on one page before Chapter 0. A timeline keyed by chapter is the lower-effort version; a two-row flowchart LR shows the braiding better.	P2

4. Chapter 0 — Orientation (01-orientation.md)

No diagrams currently. This is the chapter with the most readers who have never opened a terminal, and two of its concepts are genuinely spatial.

§	Type	Diagram	Priority
0.3.3 The Core Loop	flowchart LR	The git core loop. Working directory → (git add) → staging area → (git commit) → local repository → (git push) → GitHub, with git status shown as reading the first two and git diff reading working-directory-vs- staged. The three-places model is the single hardest thing about git for a first-time user and the prose lists commands without ever drawing the places they move things between.	P1
0.4.3 SSH Keys, Briefly	flowchart LR	The key pair. ssh-keygen produces a private key (stays in ~/.ssh/, never leaves) and a public key (uploaded to GitHub). A git push from your machine is checked against the public key GitHub holds. Explicitly mark the one arrow that never exists — the private key going to GitHub. Makes “matched pair” concrete for a reader who has never met asymmetric crypto.	P2

§	Type	Diagram	Priority
0.2.2 Navigation	flowchart TD	Absolute vs. relative paths. A small tree — <code>/ → Users</code> <code>→ robert → Documents,</code> <code>mortgage-calculator-book</code> — annotated with <code>pwd</code> output at one node, and <code>cd Documents</code> (relative) vs <code>cd</code> <code>/Users/robert/Documents</code> (absolute) as two arrows arriving at the same node from different starting points.	P3

Deliberately **not** diagrammed: 0.7.3’s project layout. The ASCII tree there is already the clearest possible rendering, and a mermaid version would be strictly worse.

5. Chapter 1 — Meet Your Coding Agent (02-meet-your-coding-agent.md)

Has one diagram (1.2, the model-basics flowchart). Three more would help, and one of them — the review loop — is a motif the rest of the book leans on.

§	Type	Diagram	Priority
1.5.1 What Pi Actually Does	flowchart LR	The agent stack. You (a prompt in a terminal) <code>→ Pi</code> (the framework: reads files, proposes diffs, runs commands with permission) <code>→</code> <code>~/.pi/agent/settings.json</code> (<code>defaultModel,</code> <code>defaultProvider</code>) <code>→</code> Ollama <code>→</code> the model’s weights on disk, and the return path: proposed diff <code>→</code> your review <code>→</code> <code>git commit</code> . 1.5.1 draws a careful distinction — “the model is Pi’s reasoning engine; Pi is the framework that lets that reasoning act on a real codebase” — that a picture nails in one look.	P1

§	Type	Diagram	Priority
1.6 (opening, or the 1.6.3 process-concept box)	flowchart TD, drawn as a cycle	The review loop. Prompt → agent proposes a diff → <i>you read it</i> → accept / correct / reject → git commit → next prompt. Explicitly a loop, returning to the start. This is the habit the book asks for in every single chapter through 13, and drawing it once at the moment it's introduced gives every later “review the diff the same way as 1.6” a picture to point back at.	P1
1.4.3 + 1.7	flowchart TD	Picking a model for your machine. Decision tree: available RAM → suggested parameter size (the 8/16/32/64GB table, as branches) → pull it → does it respond in reasonable time? → yes: continue; no: 1.7's escape hatch to OpenRouter, with “this isn't permanent” as the return edge back into the local path. Turns a table plus a section-and-a-half of prose into one self-service path.	P2

6. Chapter 2 — Writing SPEC.md (03-writing-spec.md)

§	Type	Diagram	Priority
2.2.3, in the “same contract, three altitudes” box	flowchart LR	Three altitudes. One idea — “what correct means” — written down three times: SPEC.md (Chapter 2, prose, for a human) → tests/ (Chapter 5, executable, for a test runner) → JSON Schema (Chapters 7 & 10, machine-readable, for a language model). Draw them as three renderings of the <i>same</i> source node, not as a sequence of transformations, with the audience labelled on each. This callout recurs verbatim across the book; it deserves the one diagram that makes it stick.	P1
2.2.3 Three Readers, One Document	flowchart TD	Three readers. SPEC.md at the centre, three arrows out: you-in-three-weeks, a collaborator, Pi. Optionally annotate each with what that reader needs from it. Simple, but it makes the “write for all three” instruction concrete, and it sets up 4.7.3’s and 9.6.2’s recurring point that Pi can only read what’s in the repository.	P2

7. Chapter 3 — Holding the Agent to a Standard (04-ruff.md)

Short chapter; one diagram is the right amount.

§	Type	Diagram	Priority
3.6.1 Before Every Commit	flowchart LR	The quality gate. Change made (by you or by Pi) → <code>ruff check .</code> → clean? → no: <code>ruff check --fix .</code> , then fix the rest by hand, loop back → yes → <code>ruff format .</code> → <code>pytest</code> → <code>git commit</code> . Every chapter from 6 onward ends by running exactly this and says so in one line; drawing it here once means those later one-liners have a referent.	P2
3.3.1 Linting and Formatting, One Tool	flowchart TD	Two jobs, one tool. Ruff branching into <i>lint</i> (unused imports, naming, likely bugs — E, F, I) and <i>format</i> (spacing, quotes, line length), with the note that Ruff replaces Black + Flake8 + isort. Only worth it if the chapter wants a second visual; the prose is already crisp.	P3

8. Chapter 4 — The Domain: Fixed Mortgages (05-fixed-mortgages.md)

The heaviest math chapter in the book and currently entirely unillustrated. Highest-value chapter in this plan after Chapter 11.

§	Type	Diagram	Priority
4.3 The Four Core Quantities	flowchart LR	Inputs → derived → output. principal, annual_rate, term_years, payments_per_year on the left; $r = \text{annual_rate} / \text{payments_per_year}$ and $n = \text{term_years} \times \text{payments_per_year}$ in the middle; M on the right. This exact shape reappears as SPEC.md’s “Derived quantities” section (4.7.4), as core.py’s three functions (6.4–6.5), and as MortgageInput’s fields (7.5.1) — one diagram that three later chapters can be read against.	P1
4.4.2 Where It Comes From	flowchart TD	Two directions, one end point. Left branch: “if nothing were ever paid” → principal compounds → $P(1+r)^n$. Right branch: each payment M grows for the periods remaining after it → the geometric series → $M \cdot [(1+r)^n - 1] / r$. Both arrive at a single node labelled “measured at payment n”, joined by =, with “solve for M” leading out to the formula. The derivation’s whole argument is that two things meet at one point — which is exactly what a diagram does better than a paragraph.	P1

§	Type	Diagram	Priority
4.4.3 Two Edge Cases	flowchart TD	The $r = 0$ branch. $r == 0 ? \rightarrow$ yes: $M = P / n$ (the general formula divides by r); no: the general formula. Plus a side note that $n == 1$ needs <i>no</i> branch because the general formula already produces $P(1+r)$. Reused almost verbatim as Chapter 6.5.1’s implementation, so it pays twice.	P2
4.2.3 The Shape of a Payment	xychart-beta	Interest vs. principal over the life of the loan. Two series across payments $1 \rightarrow 360$ at the worked example’s numbers: the interest portion falling, the principal portion rising, crossing somewhere past the midpoint, with the total staying flat at \$1,199.10. 4.2.3 explicitly says “understanding this shape now will make the formula feel like it’s describing something real” — and shape is the one thing prose can’t hand you. Caveat: xychart-beta is version-sensitive; verify in both builds, and fall back to a static image or drop it if the renderer here doesn’t support it.	P2

9. Chapter 5 — Tests & Test-Driven Development (06-tests-and-tdd.md)

§	Type	Diagram	Priority
5.2.1 Red, Green, Refactor	<code>stateDiagram-v2</code> (or a cyclic <code>flowchart</code>)	The TDD cycle. Red (write a failing test) → Green (smallest code that passes) → Refactor (clean up, tests protecting you) → back to Red. Annotate each transition with what has to be true to leave that state. The most-diagrammed concept in software teaching, for a good reason, and this book returns to the cycle by name in 6.8.1 and 12.5.1.	P1
5.4.2 The Worked Example as a Fixture	<code>flowchart LR</code>	How a fixture reaches a test. <code>tests/conftest.py</code> defines <code>worked_example</code> → any test function whose <i>parameter name</i> is <code>worked_example</code> receives the dict → <code>pytest</code> matches by name, no import. Show the name as the link between the two, since “ <code>pytest</code> matches it by name, no import needed” is genuinely surprising the first time and easy to gloss over. <code>capsys</code> (8.6.2) and <code>monkeypatch</code> (11.8.1) both plug into the same picture later.	P2

§	Type	Diagram	Priority
5.6.4 (the parametrize discussion)	flowchart LR	What parametrize does. One test body + a table of N rows → N separate test runs, each row's values injected by name into the signature. Would make the chapter's actual bug — a table supplying ppy/expected to a signature that names neither — visible rather than something the reader has to hold in their head across two code blocks.	P3

10. Chapter 6 — Mathematical Core Library (07-math-core-library.md)

§	Type	Diagram	Priority
6.5.1 Code That Mirrors the Math	flowchart TD	core.py's call graph. calculate_payment → annual_rate_to_periodic (→ r) and total_payments (→ n) → branch on r == 0 → principal / n or the general formula → return. Deliberately drawn to be the same shape as 4.3's and 4.4.3's diagrams, so a reader can hold the math picture and the code picture side by side — which is precisely what 6.5.1 asks them to do in prose.	P1

§	Type	Diagram	Priority
6.2 What “Pure” Means	flowchart TD	The purity fence. <code>calculate_payment</code> in a box, with what’s allowed across the boundary (arguments in, a return value out) and what isn’t (printing, files, network, global state) shown as crossed-out arrows. Only build this if §11’s layered-architecture diagram (Chapter 7) isn’t already carrying the idea; there’s real overlap.	P3

11. Chapter 7 — Validation (08-validation.md)

§	Type	Diagram	Priority
7.7.1 The Only Path In	flowchart LR	The layered architecture. Outside world → <code>MortgageInput</code> (validate) → <code>calculate_validated_payment</code> → <code>calculate_payment</code> (pure core), with a crossed-out arrow going straight from the outside world to the core. This is the spine of the entire back half of the book: Chapters 8, 9, 10, 11 and 13 each add one more thing on the left-hand side and change nothing else. Draw it here in its two-layer form, then extend the <i>same</i> diagram in 9.2, 10.2.2 and 11.9 rather than drawing a new one each time.	P1

§	Type	Diagram	Priority
7.5.1 The Model	flowchart TD	What Pydantic actually checks, in order. Raw input → unknown field present? (extra="forbid" → reject) → type coercion ("abc" → float fails) → each @field_validator in turn → a valid MortgageInput or a ValidationError carrying a human-readable message. Two outcomes, one path — and 7.5.3 makes the point that those messages are a user-facing surface, which the diagram can label directly.	P2

12. Chapter 8 — Command Line Interface (09-cli.md)

§	Type	Diagram	Priority
8.4.1 The Full Pipeline, Assembled	flowchart TD	Three chapters connected. <code>argv</code> → <code>build_parser</code> / <code>argparse</code> → <code>MortgageInput</code> → branch: <code>ValidationError</code> → message to <code>stderr</code> , exit code 1; valid → <code>calculate_validated_payment</code> → <code>calculate_payment</code> → branch on <code>--format:</code> text to <code>stdout</code> or <code>json.dumps</code> to <code>stdout</code> → exit code 0. The <code>stdout/stderr</code> split and the exit codes are both things the chapter tests for explicitly (8.6.2) and both are easy to lose in prose. 8.4.1's own sentence — “three chapters of work, connected for the first time” — is a caption waiting for a picture.	P1
8.7.3 Never Commit Your Secrets	flowchart LR	The <code>.env</code> pattern. <code>.env</code> (real key, git-ignored) and <code>.env.example</code> (placeholder, committed) both feeding the same shape; <code>load_dotenv()</code> → <code>os.getenv</code> → <code>config.py</code> → (Chapter 11) <code>OPENROUTER_API_KEY</code> . Mark the git boundary explicitly — which of the two files crosses into the repository and which never does. The rule is the whole point of the section and it's a rule about <i>where files go</i> .	P2

13. Chapter 9 — Calculator UI (10-calculator-ui.md)

§	Type	Diagram	Priority
9.3.4 Signals and Slots, or 9.7.1	flowchart LR	Click to answer, and where the testable seam is. Calculate.clicked (signal) → on_calculate (slot) → parse_form_values (text → typed values) → MortgageInput → calculate_validated_payment → result label; the except (ValueError, ValidationError) path branching to the same label with an error. Draw a dotted box around parse_form_values labelled “the part tests/test_ui.py can reach without a QApplication” — that makes 9.7.1’s extraction and 9.7.2’s decision <i>not</i> to test the rest legible in one look.	P1
9.2 Two Front Ends, One Core	flowchart LR	The spine, extended once. §11’s Chapter 7 diagram with a second entry point added: CLI and GUI both arriving at MortgageInput, everything downstream unchanged. Chapter 9’s opening claims this is where the discipline pays off; showing the <i>unchanged</i> right-hand side is the proof.	P2

Deliberately **not** diagrammed: 9.4.1’s and 9.6.1’s window sketches. Those are ASCII wireframes of a real layout and mermaid would render them worse.

14. Chapter 10 — Tool Interface (11-tool-interface.md)

§	Type	Diagram	Priority
10.3.2 What Still Needs to Be Added	flowchart TD	Anatomy of a tool definition. <code>get_tool_definition()</code> assembled from four parts, colour-split by origin: <i>free from Chapter 7</i> — parameters <code>(MortgageInput.model_json_schema(),</code> including additionalProperties: <code>false);</code> <i>new in this chapter</i> — name and description (prose, telling the model <i>when</i> to call it) and output_schema (reused from 8.5.4's <code>{"payment": ...}</code>). 10.3.1 and 10.3.2 spend two sections on exactly this “what’s reused vs. what’s new” split.	P1
10.5.1 Tests First, No Model Involved	flowchart TD	call_tool never raises. Arguments dict in → <code>MortgageInput(**arguments)</code> → branch: valid → <code>{"payment": 1199.1};</code> ValidationError → caught → <code>{"error": "..."}.</code> Mark the branch that <i>doesn't</i> exist: no exception escapes. 10.5.1 and 10.5.2 make this the chapter’s central contract, and 11.4.3 and 13.4.2 both depend on it holding.	P1

§	Type	Diagram	Priority
10.6.2 The Shape of MCP	flowchart LR	Server, client, model. An MCP <i>server</i> exposing tools (each with a name, description, schema — the same four parts as the diagram above); an MCP <i>client</i> discovering them and invoking them; the model deciding <i>which</i> and <i>when</i> . Map <code>tool.py</code> onto the server half explicitly, since 10.6.1’s whole point is “what you just built, named”. A conceptual sidebar with no code is exactly where a diagram carries the most weight.	P2

15. Chapter 11 — LLM Interface (`12-llm-interface.md`)

The longest chapter in the book, the most moving parts, and no diagram. The first row here is the highest-value single diagram in this plan.

§	Type	Diagram	Priority
11.4.1 The Request/Response Shape	sequenceDiagram	The tool-calling loop. Four participants: User, <code>llm.py</code>, Model, <code>tool.py</code>. (1) User asks a question. (2) <code>llm.py</code> → Model: the question plus the tool definition. (3) Model → <code>llm.py</code>: a <code>tool_calls</code> entry with arguments — <i>or</i> plain content, the no-call branch. (4) <code>llm.py</code> → <code>tool.py</code>: <pre>call_tool(arguments);</pre> <code>tool.py</code> → <code>llm.py</code>: <code>{"payment": ...}</code> or <code>{"error": ...}</code>. (5) <code>llm.py</code> → Model: the whole conversation replayed, with the tool result appended as a <code>tool</code> message. (6) Model → User: a plain-language answer. Show the two <code>chat</code> calls as two distinct round-trips — the “two calls, with the tool’s result inserted between them” structure 11.4.2’s comments explain three times over is the thing readers most often get wrong, and a sequence diagram is the natural form for it.	P1

§	Type	Diagram	Priority
11.9 (after the CLI and UI are wired)	flowchart TD	The whole hybrid system, once. --ask (CLI) and the Ask button (UI) → <code>llm.py</code> (<code>ask_local / ask_hosted</code>) → the model → <code>tool.py</code> → <code>MortgageInput</code> → <code>calculate_payment</code>; alongside them the structured CLI flags and the Calculate button going straight to <code>MortgageInput</code>. Every path lands on the same core. 11.1 promises “the chapter where the hybrid system this book has been building toward finally exists, end to end” — this is that claim, drawn. Reuse it in the Closing (§18).	P1
11.2.1 Explicitly Distinguishing the Roles	flowchart LR	Two models, two jobs. Left: the model behind Pi — reads your <i>code</i>, proposes diffs, helped <i>build</i> the system (Chapter 1). Right: the intelligence-engine model — reads your <i>users’ questions</i>, decides whether to call the tool, helps <i>run</i> the system (this chapter). Same technology, different job, different selection criteria (11.2.2, 11.3.1). Two things being confusable is exactly the case for putting them side by side.	P2

§	Type	Diagram	Priority
11.6.4 Wiring OpenRouter In	flowchart LR, two parallel tracks	Where the two clients actually differ. <code>ask_local</code> and <code>ask_hosted</code> drawn as parallel lanes over the same four steps, with only the three real differences highlighted: arguments arrive as a dict vs. a JSON string needing <code>json.loads</code> ; the reply needs a <code>tool_call_id</code> ; content is <code>str(result)</code> vs. <code>json.dumps(result)</code> . 11.8.2 warns these are the mistakes that “fail silently rather than crash outright” — worth making visible before the reader reviews Pi’s version.	P2

16. Chapter 12 — Evaluation (13-evaluation.md)

§	Type	Diagram	Priority
12.5.2 Scoring	flowchart TD	The eval harness. <code>data/eval_set.json</code> → <code>load_eval_set</code> → <code>run_eval</code> looping per case → <code>score_case</code> : did <code>tool_called</code> match <code>expected_tool_call</code> ? → no: fail (“ <code>tool_called</code> mismatch”); yes → were arguments expected? → compare each within 0.001 → mismatch: fail (naming the key); all match: pass → <code>summarize</code> → “6/8 passed”. The two-stage check with an early exit is the function’s actual logic and a flowchart is how it reads most clearly.	P1

§	Type	Diagram	Priority
12.2, in the process-concept box	flowchart LR, two lanes	Test vs. eval. <i>Unit test</i> : fixed input → deterministic code → exact expected output → pass/fail, every time. <i>Evaluation</i> : a question → a model whose output varies → a judgment about whether behaviour stayed in bounds → pass rate over a set. Same shape, three substituted parts. 12.2.2 says conflating them “leads to the wrong kind of disappointment in both” — side-by-side lanes make the substitution obvious.	P2
12.5.1 A Small Refactor First	flowchart LR	Wrapper delegation. <code>ask_local</code> → <code>ask_local_detailed</code> → { <code>answer</code> , <code>tool_called</code> , <code>arguments</code> }, with <code>ask_local</code> returning only [<code>"answer"</code>] so Chapter 11’s callers see no change. Small enough that prose covers it; include only if the section feels dense.	P3

17. Chapter 13 — Hardening (14-hardening.md)

Already has three (§2). Both additions below sharpen things the existing diagrams only gesture at.

§	Type	Diagram	Priority
13.4.2–13.4.3	flowchart TD	The model-output decision tree. Raw model response → tool call present? → no: content empty? → yes: the 13.4.3 fallback string; no: return the content (legitimate no-call). Tool call present → does <code>json.loads</code> succeed? → no: the 13.4.2 “trouble understanding those loan details” response, <code>arguments=None</code>, and no second model call; yes → <code>call_tool</code> → valid? → <code>{"payment": ...}</code> or <code>{"error": ...}</code>. Four distinct failure exits, each with its own graceful response — 13.3.1’s existing diagram shows <i>where</i> the seams are, this one shows <i>what happens</i> at the messiest of them. It also makes 13.4.4’s <code>call_count == 1</code> assertion self-evident rather than something the prose has to argue for.	P1
13.7.1 The Complete Definition of Done	timeline	The checklist, accumulating. Each chapter contributing its one or two permanent lines to the final list — Ch. 0 git-tracked, Ch. 3 Ruff clean, Ch. 6 core pure, Ch. 7 one path in, and so on. Would pair with the existing SPEC.md timeline in 13.6. Risk of feeling redundant next to the checklist itself, which is why it’s P3.	P3

18. Closing (15-closing.md)

§	Type	Diagram	Priority
C.2 What Got Built	flowchart TD	The finished system, one picture. The 11.9 diagram (§15) reused, extended with the two things Chapters 12 and 13 wrapped around it: the eval set checking the model-facing layer, and the logging/error handling at each seam. C.2 is a single dense paragraph naming eight chapters' worth of components and their relationships — the one paragraph in the book most improved by a picture beside it. Reusing the Chapter 11 diagram rather than drawing a new one also <i>shows</i> that nothing changed structurally after Chapter 11.	P1
C.4 Why the Boundary Sat Where It Did	flowchart TD	The boundary, revisited. The front-matter diagram (§3) redrawn with real artifacts in each zone: <i>yours</i> — SPEC.md's revisions, the \$1,199.10 worked example, every expected value, the eval set, the review checklists; <i>Pi's</i> — implementation drafts, wiring, boilerplate, the README update. The single crossing arrow now labelled with what actually crossed: artifacts Pi worked <i>from</i>, never wrote. Bookending the front-matter diagram closes the argument visually.	P2

19. Appendix (16-appendix.md)

§	Type	Diagram	Priority
A.9 Beyond This Book	flowchart TD	Where each left-out tool would slot in. The finished architecture from C.2, with the appendix's six entries pinned to the exact components they'd attach to: Docker → the CLI or tool interface as a service (A.2 says explicitly <i>not</i> the PyQt5 UI); GitHub Actions → the whole check suite on push (A.3); mypy → <code>core.py</code> and <code>tool.py</code> (A.4); pre-commit → the Chapter 3 quality gate (A.5); Typer → <code>cli.py</code> alone (A.6); PyInstaller → the GUI specifically (A.8). Every entry already names its insertion point in prose; one map turns the appendix from six independent paragraphs into a single next-project plan.	P2

20. Suggested build order

If these get added incrementally rather than all at once, this order front-loads the ones that carry the most weight and that later diagrams reuse:

1. **Ch. 7 §7.7.1 — the layered architecture.** Everything from Chapter 8 onward extends it; build it first and extend rather than redraw.
2. **Ch. 11 §11.4.1 — the tool-calling sequence.** Longest chapter, most moving parts, currently zero diagrams.
3. **Ch. 4 §4.3 and §4.4.2 — the four quantities and the two directions.** The math chapter carries the whole book's answer key and is entirely unillustrated.
4. **Ch. 8 §8.4.1 — the CLI pipeline.** First convergence of three chapters' work.
5. **Ch. 5 §5.2.1 — red/green/refactor.** Referenced by name in three later chapters.
6. **Front matter + Ch. 2 §2.2.3 + Closing C.4 — the thesis diagrams.** Build these as one set, since they're deliberately the same picture at three points.
7. **Ch. 0 §0.3.3 — the git core loop.** Highest value per line for the book's least-experienced readers.
8. Everything else, in chapter order.

Rough totals: **4 existing, 19 P1, 17 P2, 6 P3** proposed — 36 new diagrams if P1 and P2 are both built, 42 with P3, across fourteen chapters plus the front and back matter. That averages a little over two per chapter, weighted toward Chapters 4, 7, 8, 10 and 11, where the system actually gets built and where there is currently nothing at all.